EXPERIMENT 2 – GIT AND GITHUB INTEGRATION

Last-minute practical exam notes | Only essential commands

Aim: Initialize a Git repository, track and commit automation-script output, connect to GitHub, push changes using branches, and implement pre-commit and post-commit hooks.

Note: Replace **USERNAME** with your GitHub username. Create an empty GitHub repository named **AutomationProject** on GitHub first (do not add a README).

1. Install Git and configure

```
sudo apt update
sudo apt install git -y
git --version

git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
```

2. Create repository and automation script

```
mkdir -p ~/AutomationProject
cd ~/AutomationProject
git init

nano monitor.sh
```

Enter this script in nano, then save with Ctrl+O, Enter and exit with Ctrl+X:

```
#!/bin/bash
echo "System check executed at $(date)" >> system.log
echo "Automation completed"
```

```
chmod +x monitor.sh
./monitor.sh
cat system.log
```

3. Track and commit files

```
git status
git add monitor.sh system.log
git commit -m "Added automation monitoring script"
git log --oneline
```

4. Connect to GitHub – choose ONE method

First create the empty repository **AutomationProject** on github.com. The three terminal connection methods below are alternatives; you do not need to perform all three.

Method A – HTTPS with Personal Access Token (PAT)

```
git remote add origin https://github.com/USERNAME/AutomationProject.git
git branch -M main
git push -u origin main
```

At the prompts: enter your GitHub username. For Password, paste a GitHub Personal Access Token (PAT), not your GitHub account password. Create a token in GitHub Settings → Developer settings → Personal access tokens; give it access to this repository with Contents: Read and write. Keep the token private.

Method B – GitHub CLI browser login

```
sudo apt install gh -y
gh auth login
```

Choose: GitHub.com → HTTPS → Yes (authenticate Git with credentials) → Login with a web browser. Enter the one-time code in the browser and authorize. Then push:

```
git remote add origin https://github.com/USERNAME/AutomationProject.git
git branch -M main
git push -u origin main
```

Method C – SSH key

```
ssh-keygen -t ed25519 -C "your-email@example.com"
cat ~/.ssh/id_ed25519.pub
```

Copy the displayed public key. Add it on GitHub: Settings → SSH and GPG keys → New SSH key. Then connect and push:

```
git remote add origin git@github.com:USERNAME/AutomationProject.git
git branch -M main
git push -u origin main
```

Check the remote:

```
git remote -v
```

5. Create and push a branch for scheduled jobs

```
git branch scheduled-jobs
git checkout scheduled-jobs
echo "Scheduled task update" >> system.log
git add system.log
git commit -m "Scheduled job update"
git push -u origin scheduled-jobs
git branch
```

6. Git hooks

Pre-commit hook – runs before a commit:

```
nano .git/hooks/pre-commit
```

```
#!/bin/bash
echo "Checking files before commit"
```

```
chmod +x .git/hooks/pre-commit
```

Post-commit hook – runs after a successful commit and records the commit time:

```
mkdir -p ~/logs
nano .git/hooks/post-commit
```

```
#!/bin/bash
echo "Commit completed at $(date)" >> ~/logs/commit_history.log"
```

```
chmod +x .git/hooks/post-commit

echo "Testing hooks" >> test.txt
git add test.txt
git commit -m "Testing Git hooks"
cat ~/logs/commit_history.log
```

7. Automatic commit script and cron schedule

Create a script to add, commit and push changed files:

```
nano auto_commit.sh
```

```
#!/bin/bash
cd ~/AutomationProject
git add .
if ! git diff --cached --quiet; then
    git commit -m "Automatic update"
    git push origin scheduled-jobs
fi
```

```
chmod +x auto_commit.sh
./auto_commit.sh
```

Schedule it every 10 minutes:

```
crontab -e
```

Add this line (use your actual Linux home path):

```
*/10 * * * * /home/USERNAME/AutomationProject/auto_commit.sh
```

```
crontab -l
```

8. Essential verification commands

```
git status
git log --oneline
git branch
git remote -v
```

```
cat system.log
cat ~/logs/commit_history.log
crontab -l
```

Remember for the exam

• **git init** – creates a local repository.
• **git add** – stages files.
• **git commit** – saves a version locally.
• **git push** – uploads commits to GitHub.
• **branch** – separate line of development.
• **pre-commit** – executes before commit; **post-commit** – executes after commit.
• **crontab** – schedules the automation script.

Common exam mistakes: use `nano monitor.sh` (no space); run `chmod +x` before executing a script; use a PAT or SSH key for HTTPS/SSH authentication (not your GitHub account password); create `~/logs` before writing the hook log.